

第二章：数据的表示和运算

一、给出以下概念的解释说明。

真值：真值是数学上的值，即我们实际想要表示的数，不考虑计算机内部如何表示。

机器数：机器数是在计算机内部用来表示数值的二进制编码。它可能由于编码方式（如原码、补码等）和机器的特性而有所不同。

数值数据：数值数据是指那些可以表示数量或大小的数据，如整数、浮点数等。

非数值数据：非数值数据是指不能表示为数量或大小的数据，如字符、文本、图片等。

无符号整数：无符号整数是一个非负的二进制数，它只表示大小，不表示正负。

带符号整数：带符号整数可以表示正数或负数。在计算机中，通常使用最高位作为符号位，其余位表示数值大小。

定点数：定点数是一种数值表示方法，其中小数点位置是固定的。定点数主要用于表示整数或小数。

浮点数：浮点数是一种数值表示方法，其中小数点位置可以浮动。它通常由尾数（mantissa）和阶码（exponent）两部分组成，用于表示非常大或非常小的数。

尾数：尾数是浮点数中用于表示数值大小的部分。

阶和阶码：阶是浮点数中小数点实际位置与默认位置之间的偏移量。阶码则是这个偏移量的编码表示。

溢出：溢出是指计算机在运算过程中，结果超出了它能表示的范围。这可能导致不正确的结果或错误。

规格化数：规格化数是指满足特定条件的浮点数，例如尾数的最高位是 1（对于二进制数）。这种表示方式可以提高浮点数的精度和范围。

左规：左规是在浮点数运算中，当尾数向左移动时，阶码相应减小的过程。这通常发生在尾数溢出时。

右规：右规是在浮点数运算中，当尾数向右移动时，阶码相应增加的过程。这通常发生在尾数下溢时。

ASCII 码：ASCII（American Standard Code for Information Interchange）码是一种用于表示英文字符和其他控制字符的编码系统。

汉字输入码：汉字输入码是用户通过键盘等设备输入汉字时所使用的编码，如拼音、五

笔等。

汉字内码：汉字内码是计算机内部用于表示汉字的编码，如 GB2312、GBK、UTF-8 等。

字长：字长是指计算机内部处理数据的位数，例如 32 位或 64 位处理器。它决定了计算机一次能处理的数据量。

大端方式：大端方式（Big Endian）是一种数据存储方式，其中高位字节存储在内存的低地址端，低位字节存储在内存的高地址端。

小端方式：小端方式（Little Endian）是另一种数据存储方式，其中低位字节存储在内存的低地址端，高位字节存储在内存的高地址端。

ALU（算术逻辑单元）：ALU 是计算机中的一个重要组件，负责执行算术运算（如加、减、乘、除）和逻辑运算（如与、或、非）。

2.简单回答下列问题。

(1)为什么计算机内部采用二进制表示信息？

解析：二进制只有 0 和 1 两种状态，这与计算机中的逻辑电路的开关状态一致，使得信息表示和计算非常直接和简单。

二进制数的运算规则简单，使得计算机在进行运算时更加高效。

二进制数的抗干扰能力强，因为它只涉及两种状态，对噪声和干扰的容忍度较高。

(2)既然计算机内部所有信息都用二进制表示，为什么还要用到十六进制或八进制数？

解析：虽然计算机内部所有信息都用二进制表示，但使用十六进制或八进制数是为了方便人类理解和操作。十六进制和八进制数的每一位可以对应多个二进制位，这使得在表示较长的二进制数时，十六进制或八进制数更加简洁，便于阅读和书写。在程序设计和调试中，使用十六进制或八进制数可以更方便地查看和操作内存地址和数据。

(3) 在浮点数的基数和总位数一定的情况下，浮点数的表示范围和精度分别由什么决定？

解析：在浮点数的基数和总位数一定的情况下，浮点数的表示范围和精度分别由阶码和尾数决定。阶码决定了浮点数的范围，即能表示的最大和最小数值；而尾数决定了浮点数的精度，即数值的精确度或小数部分的位数。

(4)为什么要对浮点数进行规格化？有哪两种规格化操作？

解析：对浮点数进行规格化是为了提高浮点数的精度和运算效率。规格化操作包括左规和右规两种。当尾数的绝对值小于规定的最小值时，需要进行右规操作，即将尾数的小数点向右移动，同时增加阶码的值；当尾数的绝对值大于规定的最大值时，需要进行左规操作，

即将尾数的小数点向左移动，同时减少阶码的值。通过规格化操作，可以确保浮点数的尾数满足一定的条件（如最高位为1），从而提高浮点数的表示范围和精度。

(5)为什么计算机处理汉字时会涉及不同的编码(如输入码、内码、字模码)?说明这些编码中哪些用二进制编码，哪些不用二进制编码，为什么?

解析：计算机处理汉字时会涉及不同的编码，如输入码、内码和字模码，是因为汉字数量庞大，需要有效的编码方式来存储、传输和显示。这些编码中，输入码如拼音码、五笔码等不是用二进制编码的，因为它们是为了方便用户输入汉字而设计的，具有特定的输入规则和逻辑。而内码和字模码则是用二进制编码的。内码是计算机内部用于存储和传输汉字的唯一标识码，需要二进制编码来确保唯一性和计算机识别的准确性。字模码是用于显示汉字的字形信息，通常以二进制图像或点阵形式表示，因此也需要二进制编码。

3.实现下列各数的转换

(1) (25.8125)₁₀= (?)₂= (?)₈= (?)₁₆

解析：(25.8125)₁₀= (11001.1101)₂= (31.64)₈= (19)₁₆

十进制到2进制转换：

将目标基数乘以结果的小数位(最高可达6位数):
floor(25.8125×2⁴) = 413

除以基数，从余下的数字中得到数字。

除法	商数	余数	数位
413/2	206	1	0
206/2	103	0	1
103/2	51	1	2
51/2	25	1	3
25/2	12	1	4
12/2	6	0	5
6/2	3	0	6
3/2	1	1	7
1/2	0	1	8

= (11001.1101)₂

十进制到8进制转换：

将目标基数乘以结果的小数位(最高可达6位数):
floor(25.8125×8²) = 1652

除以基数，从余下的数字中得到数字。

除法	商数	余数	数位
1652/8	206	4	0
206/8	25	6	1
25/8	3	1	2
3/8	0	3	3

= (31.64)₈

(2) (101101.011)₂= (?)₁₀= (?)₈= (?)₁₆

解析：(101101.011)₂= (45.375)₁₀= (55.3)₈= (2D.6)₁₆

2进制到十进制转换： $(101101.011)_2 = (1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2}) + (1 \times 2^{-3}) = (45.375)_{10}$

(3) (4E.C)₁₆= (78.75)₁₀= (1001110.11)₂

16进制到十进制转换:

$$(4E.C)_{16} = (4 \times 16^1) + (14 \times 16^0) + (12 \times 16^{-1}) = (78.75)_{10}$$

4.假定机器数为 8 位(1 位符号, 7 位数值), 写出下列各二进制小数的原码表示。

+0.1001, -0.1001, +1.0, -1.0, +0.010100, -0.010100, +0, -0

解析:

给定机器数为 8 位, 其中 1 位用于符号, 7 位用于数值。原码表示法非常直观, 正数的符号位为 0, 负数的符号位为 1, 其余位直接表示数值。以下是各二进制小数的原码表示:

+0.1001

- 符号位: 0 (正数)
- 数值位: 1001
- 原码表示: 01001000

-0.1001

- 符号位: 1 (负数)
- 数值位: 1001
- 原码表示: 11001000

+1.0

- 符号位: 0 (正数)
- 数值位: 因为小数点后的位数不足 7 位, 所以需要在前面补 0 直到达到 7 位。因此, 数值位为 0000000 (表示 1.0)
- 原码表示: 00000000

-1.0

- 符号位: 1 (负数)
- 数值位: 与+1.0 相同, 为 0000000
- 原码表示: 10000000

+0.010100

- 符号位: 0 (正数)
- 数值位: 010100
- 原码表示: 00101000

-0.010100

- 符号位: 1 (负数)

- 数值位：010100

- 原码表示：10101000

+0

- 符号位：0（正数）

- 数值位：全 0，表示 0

- 原码表示：00000000

-0

- 符号位：1（负数）

- 数值位：全 0，但在原码表示法中，-0 和+0 是相同的，都表示为 00000000

- 原码表示：00000000（注意，在实际计算机系统中，-0 通常会被特殊处理或视为非法表示）

请注意，原码表示法有一个问题，那就是它有两个 0 的表示（+0 和-0）。在实际的计算机系统中，通常会使用补码或其他方法来解决这个问题。

5.假定机器数为 8 位(1 位符号，7 位数值)，写出下列各二进制整数的补码和移码表示。

+1001,-1001,+1,-1,+ 10100,-10100, +0,-0

对于 8 位机器数（1 位符号，7 位数值），我们可以按照补码和移码的规则来求出各二进制整数的表示。

首先，补码的定义是：正数的补码与其原码相同；负数的补码是其绝对值的二进制表示（即原码），然后各位取反，最后加 1。

移码的定义是：在补码的基础上，符号位取反（即正数符号位为 1，负数符号位为 0）。

现在，我们根据这些规则来求出各二进制整数的补码和移码表示：

+1001

- 补码：与原码相同，为 01001

- 移码：符号位取反，为 11001

-1001

- 补码：先求原码（11001），然后取反得 00110，再加 1 得 00111

- 移码：符号位取反，为 00111

+1

- 补码：00001

- 移码：10001

-1

- 补码：原码为 10001，取反得 01110，再加 1 得 01111

- 移码：01111

+10100

- 补码：010100

- 移码：110100

-10100

- 补码：原码为 110100，取反得 001011，再加 1 得 001100

- 移码：001100

+0

- 补码：0000000

- 移码：1000000

-0

- 在补码和移码表示中，通常没有单独的-0表示。在补码系统中，-0和+0是等价的，都表示为全0（0000000）。在移码系统中，由于符号位取反，-0的移码表示会是1000000，但这通常不被视为一个有效的数值表示，因为移码主要用于浮点数的表示，其中-0和+0在大多数系统中被视为相同。

请注意，对于补码和移码，符号位都包含在总的位数中，所以8位表示包括了1位符号位和7位数值位。对于+0和-0，补码系统中它们都是0000000，但在移码系统中，-0没有独立的表示，通常被视为无效或等同于+0的移码表示。

6. 已知 $[x]_{\text{补}}$ ，求 x

(1) $[x]_{\text{补}} = 11100111$

(2) $[x]_{\text{补}} = 1000\ 000$

(3) $[x]_{\text{补}} = 0101\ 0010$

(4) $[x]_{\text{补}} = 1101\ 0011$

解析：

已知补码求原码，我们需要首先确定该补码表示的是正数还是负数，然后根据补码的定义还原出原码。补码的最高位（符号位）为0时，表示正数，此时补码就是原码；补码的最高位为1时，表示负数，此时我们需要对补码求反再加1得到原码。下面针对每一个补码，我们来进行计算：

(1) $[x]_{\text{补}} = 11100111$

- 符号位为 1，表示这是一个负数。我们需要对补码求反再加 1 得到原码。
- 求反：00011000
- 加 1：00011001
- 因此， $x = -00011001$ ，即 -23（十进制）。

(2) $[x]_{\text{补}} = 10000000$

- 符号位为 1，表示这是一个负数。但这里有个特殊情况，补码表示的是-128。在 8 位补码系统中，-128 没有对应的正数形式，因此这是一个特殊的值。
- 因此， $x = -128$ （十进制）。

(3) $[x]_{\text{补}} = 01010010$

- 符号位为 0，表示这是一个正数。补码就是原码。
- 因此， $x = 01010010$ ，即 82（十进制）。

(4) $[x]_{\text{补}} = 11010011$

- 符号位为 1，表示这是一个负数。我们需要对补码求反再加 1 得到原码。
- 求反：00101100
- 加 1：00101101
- 因此， $x = -00101101$ ，即 -43（十进制）。

7. 假定 32 位字长的机器中带符号整数用补码表示，浮点数用 IEEE 754 标准表示，寄存器 R1 和 R2 的内容分别为 R1: 00000H, R2: 8080 00 不同指令对寄存器进行不同的操作，因而，不同指令执行时寄存器内容对应的真值不同。假定执行下列运算指令时,操作数为寄存器 R1 和 R2 的内容，则 R1 和 R2 中操作数的真值分别为多少？

(1)无符号数加法指令

(2)带符号整数乘法指令

(3)单精度浮点数减法指令

解析：

首先，我们需要明确每个指令如何处理寄存器中的内容。寄存器 R1 和 R2 中的内容分别是 00000H 和 80800000H，这些都是十六进制表示。

(1) 无符号数加法指令

- 对于无符号数加法，我们直接将两个数相加。
- R1: 00000000H

- R2: 80800000H
- 相加得:
- 结果 = 80800000H
- 因此, R1 和 R2 中操作数的真值(十进制表示)分别是:
- R1 真值 = 0
- R2 真值 = 2147483648 (因为 80800000H 转换为十进制是 2147483648)

(2) 带符号整数乘法指令

- 对于带符号整数乘法, 首先需要将十六进制数转换为补码表示的带符号整数。
- R1: 00000000H -> 真值 = 0
- R2: 80800000H -> 真值 = -2147483648 (因为是补码表示的负数)
- 然后, 执行乘法运算:
- $0 * (-2147483648) = 0$
- 所以, 尽管乘法运算执行了, 但结果的真值仍然是 0。

(3) 单精度浮点数减法指令

- 对于单精度浮点数减法, 我们需要按照 IEEE 754 标准来解释寄存器中的内容。
- R1: 00000000H -> 0.0 (正零)
- R2: 80800000H -> -Inf (负无穷大, 因为指数位全为 1 且尾数位不全为 0 时表示 NaN 或特殊值, 这里的尾数位为 0, 所以是负无穷大)
- 在 IEEE 754 标准中, 0 减去负无穷大在数学上是没有定义的, 但在大多数实现中, 这样的操作通常会导致结果为正无穷大。但需要注意的是, 这取决于具体的硬件和浮点实现。
- 如果我们按照通常的实现, 结果应该是正无穷大。但由于这取决于具体的实现, 所以严格来说我们不能确定 R1 和 R2 中操作数的真值在执行单精度浮点数减法指令后的确切结果, 只能说结果可能是正无穷大。

7. 假定一台 32 位字长的机器中带符号整数用补码表示, 浮点数用 IEEE 754 标准表示, 寄存器 R1 和 R2 的内容分别为 R1: 00 00 10 8BH, R2: 80 80 10 8BH。不同指令对寄存器进行不同的操作, 因而, 不同指令执行时寄存器内容对应的真值不同。假定执行下列运算指令时, 操作数为寄存器 R1 和 R2 的内容, 则 R1 和 R2 中操作数的真值分别为多少?

- (1) 无符号数加法指令 R1:+108BH R2:+8080108BH
- (2) 带符号整数乘法指令 R1:+108BH R2:-7F7FEF75H

(3) 单精度浮点数减法指令 R1:0000000000000000f 0001000010001011=+0.000000f
0001000010001011B2⁻¹²⁶

R2:100000000100000000001000010001011=-0.0000000001000010001011B2⁻¹²⁶

9. 以下是一个 C 语言程序，用来计算一个数组 a 中每个元素的和。当参数 len 为 0 时，返回值应该是 0，但是在机器上执行时，却发生了存储器访问异常。请问这是是什么原因造成的，并说明程序应该如何修改。

```
float sum_elements(float a[], unsigned len)
{
    int i;

    float result = 0;

    for (i = 0; i <= len - 1; i++)

        result += a[i];

    return result;
}
```

解：参数 len 的类型是 unsigned，当 len=0 时，len-1 的结果为 11...1，因此循环体被不断执行，引起数组越界。

应将 len 修改为 int 型或者 for 循环条件改为 i<len

10. 下列几种情况所能表示的数的范围是什么？

(1) 16 位无符号整数

(2) 16 位原码定点小数

(3) 16 位补码定点整数

(4) 下述格式的浮点数(基数为 2，移码的偏置常数为 128，没有隐藏位)

数符（1 位）阶码（8 位移码）尾数（7 位原码数值部分）

解答：对于每种情况，我们可以分别计算出其表示的数的范围。

(1) 16 位无符号整数：在 16 位的情况下，无符号整数的范围为 0 到 $2^{16} - 1$ ，即 0 到 65535。

(2) 16 位原码定点小数：16 位原码定点小数通常采用一个比特表示符号位，剩余比特表示整数和小数部分。假设有 1 位符号位，15 位用于整数和小数部分。那么范围可以计算为从 -1（全部为 1 的补码）到 $1 - 2^{-(15)}$ （最大正数）。所以表示的范围是从 -1 到 $1 - 2^{-(15)}$ 。

(3) 16 位补码定点整数：16 位补码定点整数同样采用一个比特表示符号位，剩余比特表示整数部分。假设有 1 位符号位，15 位用于整数部分。那么范围可以计算为从 -2^{15} （全

部为 1 的补码) 到 $2^{15} - 1$ (全部为 0 的补码)。所以表示的范围是从 -2^{15} 到 $2^{15} - 1$ 。

(4) 下述格式的浮点数: 这个浮点数格式有 1 位符号位, 8 位阶码 (移码), 7 位尾数 (原码数值部分)。阶码的偏置常数为 128, 没有隐藏位。对于阶码, 我们将其看作无符号数。假设阶码的偏置常数为 128, 则阶码的范围是从 0 到 255。尾数的范围是从 0 到 127。因此, 这个浮点数的表示范围是从 $2^{-(128)}$ 到 $(1 - 2^{-(7)}) * 2^{(127)}$ 。

11. 以 IEEE 754 单精度浮点数格式表示下列十进制数。+1.625, -9/16, +1.75, +19, -1/8, 258

(1) ****+1.625****:

- 首先将 +1.625 转换为二进制。整数部分是 1, 小数部分是 0.625。
- 整数部分的二进制是 '1'。
- 小数部分的二进制是 '0.101' (因为 $0.625 = 1/2 + 1/8$)。
- 因此, +1.625 的二进制表示是 '1.101'。
- 将其规范化为科学计数法形式 ' $1.101 * 2^0$ '。
- 在 IEEE 754 格式中, 符号位为 '0', 阶码为 '127' (因为指数偏移是 127, 所以阶码为 $0 + 127 = 127$), 尾数部分为 '101 0000 0000 0000 0000'。
- 因此, +1.625 的 IEEE 754 单精度浮点数表示为 '0 01111111 101000000000000000000000'。

(2) **** -9/16 ****:

- 将 -9/16 转换为二进制。 -9 转换为二进制是 '1001', 16 转换为二进制是 '10000'。
- -9/16 的二进制是 '1001.0000'。
- 将其规范化为科学计数法形式 ' $1.001 * 2^{-3}$ '。
- 在 IEEE 754 格式中, 符号位为 '1', 阶码为 '130' ($3 + 127 = 130$), 尾数部分为 '001000000000000000000000'。

- 因此, -9/16 的 IEEE 754 单精度浮点数表示为 '1 10000010 001000000000000000000000'。

(3) ****+1.75****:

- 将 +1.75 转换为二进制。整数部分是 1, 小数部分是 0.75。
- 整数部分的二进制是 '1'。
- 小数部分的二进制是 '0.11' (因为 $0.75 = 1/2 + 1/4$)。

- 因此, $+1.75$ 的二进制表示是 1.11 。
- 将其规范化为科学计数法形式 $1.11 * 2^0$ 。
- 在 IEEE 754 格式中, 符号位为 0 , 阶码为 127 ($0 + 127 = 127$), 尾数部分为 110000000000000000000000 。

- 因此, $+1.75$ 的 IEEE 754 单精度浮点数表示为 $0\ 01111111\ 110000000000000000000000$ 。

(4) $**+19**$:

- 将 $+19$ 转换为二进制。 19 的二进制是 10011 。
- 将其规范化为科学计数法形式 $1.0011 * 2^4$ 。
- 在 IEEE 754 格式中, 符号位为 0 , 阶码为 131 ($4 + 127 = 131$), 尾数部分为 001100000000000000000000 。

- 因此, $+19$ 的 IEEE 754 单精度浮点数表示为 $0\ 1000011\ 001100000000000000000000$ 。

(5) $** - 1/8 **$:

- 将 $-1/8$ 转换为二进制。 $-1/8$ 的二进制是 1111.0 。
- 将其规范化为科学计数法形式 $1.111 * 2^0$ 。
- 在 IEEE 754 格式中, 符号位为 1 , 阶码为 127 ($0 + 127 = 127$), 尾数部分为 111000000000000000000000 。

- 因此, $-1/8$ 的 IEEE 754 单精度浮点数表示为 $1\ 01111111\ 111000000000000000000000$ 。

(6) $**258**$:

- 将 258 转换为二进制。 258 的二进制是 100000010 。
- 将其规范化为科学计数法形式 $1.00000010 * 2^8$ 。
- 在 IEEE 754 格式中, 符号位为 0 , 阶码为 135 ($8 + 127 = 135$), 尾数部分为 000000100000000000000000 。

- 因此, 258 的 IEEE 754 单精度浮点数表示为 $0\ 10000111\ 000000100000000000000000$ 。

12. 设一个变量的值为 4098, 要求分别用 32 位补码整数和 IEEE 754 单精度浮点格式表示该变量(结果用十六进制表示), 并说明哪段二进制序列在两种表示中完全相同, 为什么会相同?

解答: 首先, 让我们分别用 32 位补码整数和 IEEE 754 单精度浮点格式来表示变量值 4098。

在 IEEE 754 单精度浮点格式中，符号位为 1（表示负数），阶码为 $31+127=158$ ，尾数部

分为 000000000000000000000000。

因此，IEEE 754 单精度浮点格式表示为 `1 10011110 000000000000000000000000`。

现在，让我们来分析哪种表示更精确：

- 32 位补码整数表示的值为精确值，因为在整数表示中，每个位都被精确地用于表示数值。

- IEEE 754 单精度浮点格式表示的值为近似值，因为在浮点表示中，精度是受限的，尤其是对于较大的数值，可能会丢失精度或者产生舍入误差。

因此，32 位补码整数表示的值是完全精确的，而 IEEE 754 单精度浮点格式表示的值是近似值。

16.假设某字长为 8 位的计算机中，带符号整数变量采用补码表示，已知 $x=-60$ ， $y=-75$ ， x 和 y 分别存放在寄存器 A 和 B 中。请回答下列问题(要求最终用十六进制表示二进制序列)。

(1)寄存器 A 和 B 中的内容分别是什么？

(2) 若 x 和 y 相加后的结果存放在寄存器 C 中，则寄存器 C 中的内容是什么？运算结果是否正确？加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

(3)若 x 和 y 相减后的结果存放在寄存器 D 中，则寄存器 D 中的内容是什么？运算结果是否正确？此时，加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

(4)对于带符号整数的减法运算，能否根据借位标志 CF 对两个带符号整数的大小进行比较？

解析：首先，让我们按照题目中所述的要求来回答问题。

(1) 寄存器 A 和 B 中的内容分别是什么？

由于题目中已经说明了采用 8 位补码表示，我们可以将 -60 和 -75 转换为 8 位补码表示。

-60 的补码为：`11000100`

-75 的补码为：`10110101`

所以寄存器 A 中的内容是 `11000100`，寄存器 B 中的内容是 `10110101`。

(2) 若 x 和 y 相加后的结果存放在寄存器 C 中，则寄存器 C 中的内容是什么？运算结果是否正确？加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

要将-60和-75相加，我们使用补码的加法规则。在8位补码加法中，如果最高位的进位与次高位的进位不同，则发生溢出。

所以，将-60和-75相加，得到的结果是-135。它的补码是`10001001`。

寄存器C中的内容是`10001001`。

- 运算结果是否正确？正确，因为补码加法给出了正确的结果。
- 加法器最高位的进位 Cout 是什么？由于最高位进位与次高位进位相同，所以进位 Cout 为 0。
- 溢出标志 OF 是什么？由于溢出，所以溢出标志 OF 为 1。
- 符号标志 SF 是什么？由于结果为负数，所以符号标志 SF 为 1。
- 零标志 ZF 是什么？由于结果不为零，所以零标志 ZF 为 0。

(3) 若 x 和 y 相减后的结果存放在寄存器 D 中，则寄存器 D 中的内容是什么？运算结果是否正确？此时，加法器最高位的进位 Cout 是什么？溢出标志 OF、符号标志 SF 和零标志 ZF 各是什么？

要将-60和-75相减，我们使用补码的减法规则。在8位补码减法中，如果最高位的进位与次高位的进位相同，则发生溢出。

所以，将-60和-75相减，得到的结果是15。它的补码是`00001111`。

寄存器D中的内容是`00001111`。

- 运算结果是否正确？正确，因为补码减法给出了正确的结果。
- 加法器最高位的进位 Cout 是什么？由于最高位进位与次高位进位相同，所以进位 Cout 为 0。
- 溢出标志 OF 是什么？由于没有溢出，所以溢出标志 OF 为 0。
- 符号标志 SF 是什么？由于结果为正数，所以符号标志 SF 为 0。
- 零标志 ZF 是什么？由于结果不为零，所以零标志 ZF 为 0。

(4) 对于带符号整数的减法运算，能否根据借位标志 CF 对两个带符号整数的大小进行比较？

在补码表示中，如果两个数相减，且结果为正数，则第一个数大于第二个数；如果结果为负数，则第一个数小于第二个数；如果结果为零，则两个数相等。因此，可以根据减法的结果来比较两个带符号整数的大小。CF 标志位会反映是否发生了借位，但它本身并不能直接用于比较大小。